

TDR-W: A Weighted Framework for Prioritizing Technical Debt Based on Development Pain and Static Costs

Ban Mihai-Constantin
-
-
Cluj, Romania
ban.mihaiconstantin@gmail.com

2nd Author Name
Department
Organization
City, Country
email@example.com

Abstract—This report presents a new method, called the TDR-W framework, for prioritizing technical debt (TD). It is designed to work better than existing methods, which often fail to correctly prioritize debt due to a focus on theoretical aspects rather than practical impact. The core concept is that debt risk is a combination of the effort required to fix it (the principal) and the pain it currently causes (the interest). This idea is defined with a weighted formula: $TDR_Score = (Effort) \times (w_{commits} \cdot Recent_Commits + w_{bugs} \cdot Recent_Bugs)$. This paper provides a methodology for gathering these metrics by mining a project's coding history using tools like Lizard and PyDriller, and a heuristic approach (HBFM) for identifying bugs. A case study on several major open-source projects demonstrates how this formula identifies real problem areas (hotspots) and safely deprioritizes benign debt (messy code that is not negatively impacting the team).

Index Terms—Technical Debt, Software Metrics, Risk Management, Mining Software Repositories, Prioritization.

I. INTRODUCTION

The primary motivation for a new framework is the inadequacy of current tools and metrics in predicting real-world problems. Numerous studies attempting to link code messiness (measured by tools like SonarQube) to developer outcomes (like lead time) have yielded weak or inconsistent results. In some cases, messy code had no meaningful impact on delivery speed, suggesting that the cost of fixing debt (principal) is distinct from the pain it causes (interest).

Existing prioritization methods are often overly complex or fail to address the fundamental trade-off between fixing debt and building new features. A systematic literature review found that no consensus exists on a prioritization model, with one review identifying 53 unique factors used by different researchers [4]. Furthermore, few methods account for team resource constraints.

To address this, the TDR-W framework is proposed as a simple, contextual model that measures both static "cost" and real-world "pain." This paper addresses the following research questions:

- **RQ1:** Can a simplified, weighted model that combines static costs with dynamic "pain" metrics effectively identify critical debt hotspots?

- **RQ2:** Can such a model successfully identify and deprioritize "benign debt" (high effort, low pain)?

II. THE TDR-W MODEL

A. Conceptual Framework: Principal and Interest

The TDR-W model utilizes a financial analogy:

- **The Principal (Effort to Fix):** The one-time cost to remove the debt.
- **The Interest Rate (Pain Score):** The ongoing cost of inaction, manifested as wasted time, developer friction, and customer-facing bugs.

The Total Debt Risk (TDR) score is calculated by multiplying these factors. This multiplication naturally acts as a filter:

- 1) **High-Effort, Low-Pain:** Old, complex modules that are rarely touched yield a low TDR score (Benign Debt).
- 2) **Low-Effort, High-Pain:** Simple but buggy code that changes daily yields a high TDR score (Critical Hotspot).

B. The Weighted Formula

The formal TDR-W formula is defined as:

$$TDR = E_{fix} \times (w_c \cdot C_{recent} + w_b \cdot B_{recent}) \quad (1)$$

Where:

- E_{fix} is the Effort to Fix (Principal).
- C_{recent} is Recent Commits (Friction proxy).
- B_{recent} is Recent Bugs (Unreliability proxy).
- w_c and w_b are weights for friction and unreliability, respectively.

These weights allow companies to adjust the model to their strategy. For example, a mature enterprise might weigh bugs higher ($w_b = 2.0$) than commits ($w_c = 0.5$) to prioritize stability over speed.

C. Justification of Components

The components are grounded in academic research:

- **Effort to Fix:** Standard practice in TD management [7].
- **Recent Commits:** Based on findings that debt in frequently changed code causes significant waste (up to 23% of development time) [8], [2].
- **Recent Bugs:** Supported by MacCormack & Sturtevant [13], who linked architectural coupling to defect density.

III. METHODOLOGY

This section outlines the Mining Software Repositories (MSR) approach used to instrument the model.

A. Instrumentation 1: Lizard Effort Proxy (LEP)

Most tools provide complexity metrics rather than time estimates. We propose the Lizard Effort Proxy (LEP) to estimate effort:

$$LEP = \sum NLOC + (k \times \sum CCN) \quad (2)$$

Where $\sum NLOC$ is the sum of lines of code, $\sum CCN$ is the total Cyclomatic Complexity Number, and k is a complexity penalty (e.g., $k = 5$). This aligns with research linking refactoring effort to complexity [7].

B. Instrumentation 2: Recent Commits

Using the PyDriller library, we count the number of changes to a module within a specific window (e.g., 6 months) to proxy development friction.

C. Instrumentation 3: Heuristic-Based Fix-Mapping (HBFM)

While the SZZ algorithm is the academic standard for linking bugs to commits, it is complex and often inaccurate. We utilize Heuristic-Based Fix-Mapping (HBFM). This method scans commit messages for bug-fix patterns to identify "hotspots" of unreliability. It skips the "trace-back" step of SZZ, focusing only on where fixes are currently occurring.

IV. CASE STUDY: REAL-WORLD DATA ANALYSIS

The framework was applied to seven open-source repositories: `apache/commons-cli`, `apache/commons-lang`, `psf/requests`, `zxing/zxing`, `pallets/flask`, `facebook/react`, and `chartjs/Chart.js`.

A. Data Gathering

Data was generated using the methodology in Section III, with weights set to $w_c = 0.5$ and $w_b = 1.0$, implying that a bug is twice as impactful as general development friction.

B. Results: TDR Hotspot Analysis

Table I displays the top 10 highest-risk modules across the combined dataset.

TABLE I
TOP 10 TDR-W HOTSPOTS ACROSS ALL PROJECTS

Proj	Module	TDR	Effort	Pain
React	react-reconciler/src	6.0M	59,354	101.5
React	react-server/src	3.1M	22,639	140.0
Lang	commons-lang3	2.7M	31,632	87.5
React	babel-plugin/HIR	844K	10,696	79.0
React	react-client/src	762K	9,902	77.0
React	fiber	645K	13,724	47.0
React	devtools/Views	461K	6,190	74.5
React	react-dom-bindings	341K	18,949	18.0
CLI	commons-cli	315K	6,133	51.5
React	babel-plugin/Valid	293K	5,581	52.5

C. Findings

Finding 1: Effort alone is misleading. In `commons-lang`, the `builder` module had high Effort (7,764) but low Pain (8.0), while `function` had lower Effort (1,501) but high Pain (30.5). TDR-W correctly identifies `function` as a significant risk despite its smaller size.

Finding 2: Identifying Hotspots. The `react-server` module (Rank 2) highlights the model's value. Despite having half the Effort of Rank 1, its Pain Score of 140.0 (driven by 58 recent bugs) makes it a critical priority.

Finding 3: Ignoring Benign Debt. Table II demonstrates modules with high effort but negligible pain. For instance, `zxing/oned` has an effort of 7,187 but a Pain Score of only 0.5. TDR-W deprioritizes this, allowing teams to focus on active fires.

TABLE II
EXAMPLES OF BENIGN DEBT (HIGH-EFFORT, LOW-PAIN)

Project	Module	Effort	Pain	TDR
zxing	core/oned	7,187	0.5	3,593
Chart.js	plugin.filler	1,282	0.5	641
Lang	lang3/exception	1,188	0.5	594

V. DISCUSSION AND FUTURE WORK

A. Implications for Practitioners

TDR-W serves as a quantitative engine for a "Prioritization Matrix." By plotting TDR_Score (Risk) against Business_Value, teams can solve the "debt vs. features" dilemma. Items with High TDR and High Business Value become immediate priorities.

B. Limitations

- **Effort Proxy:** The LEP model is an unvalidated heuristic.
- **Bug Detection:** HBFM relies on commit message quality and can be prone to false positives/negatives (e.g., "fixed typo" vs "fixed bug").
- **Subjectivity:** Weights (w_c, w_b) are strategic choices rather than objective constants.

C. Future Work

Future research should focus on validating the LEP formula against human estimates of refactoring time. Additionally, the bug-finding script should be improved to utilize issue-tracker APIs (Jira/GitHub) to verify issue types, reducing false positives. Finally, a longitudinal A/B test in an industrial setting is required to measure the impact of TDR-W on lead time and defect rates compared to traditional prioritization methods.

REFERENCES

- [1] D. Saraiva, J. G. Neto, U. Kulesza, G. Freitas, R. Reboucas, and R. Coelho, "Technical Debt Tools: A Systematic Mapping Study," in *Proceedings of the 23rd International Conference on Enterprise Information Systems (ICEIS 2021)*, vol. 2, pp. 88-98, SCITEPRESS, 2021.
- [2] K. Gupta, "Measuring the Impact of Technical Debt on Development Effort in Software Projects," Research Paper/Proposal, 2018.
- [3] B. Paudel, J. Gonzalez-Huerta, E. Zabardast, and E. Klotins, "Towards Measuring the Impact of Technical Debt on Lead Time: An Industrial Case Study," arXiv preprint arXiv:2406.01578, 2024.
- [4] R. Alfayez, W. Alwehaibi, R. Winn, E. Venson, and B. Boehm, "A Systematic Literature Review of Technical Debt Prioritization," in *International Conference on Technical Debt (TechDebt '20)*, pp. 1-10, ACM, 2020.
- [5] M. Wiese, K. Serwa, A. Besier, A. S. Marion-Jetten, and E. Bittner, "Establishing Technical Debt Management: A Five-Step Workshop Approach and an Action Research Study," arXiv preprint arXiv:2508.15570, 2025.
- [6] N. Brown, Y. Cai, Y. Guo, R. Kazman, et al., "Managing technical debt in software-reliant systems," *FSE/SDP workshop on Future of software engineering research*, pp. 47-52, 2010.
- [7] C. Seaman and Y. Guo, "Measuring and monitoring technical debt," *3rd international workshop on managing technical debt (MTD)*, pp. 1-7, 2011.
- [8] T. Besker, A. Martini, and J. Bosch, "Technical debt cripples software developer productivity: A longitudinal study on developers' daily software development work," *Proceedings of the 2018 International Conference on Technical Debt*, pp. 105-114, 2018.
- [9] V. Lenarduzzi, T. Besker, D. Taibi, A. Martini, and F. A. Fontana, "A systematic literature review on Technical Debt prioritization: Strategies, processes, factors, and tools," *Journal of Systems and Software*, vol. 171, 110827, 2021.
- [10] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, pp. 2503-2511, 2015.
- [11] J. D. Morgenthaler, M. Gridnev, R. Sauciuc, and S. Bhansali, "Searching for build debt: Experiences managing technical debt at Google," in *Proceedings of the 3rd International Workshop on Managing Technical Debt (MTD)*, pp. 1-6, IEEE, 2012.
- [12] T. Besker, A. Martini, and J. Bosch, "Technical debt cripples software developer productivity: A longitudinal study on developers' daily software development work," in *Proceedings of the 2018 International Conference on Technical Debt*, pp. 105-114, 2018.
- [13] A. MacCormack and D. J. Sturtevant, "Technical debt and system architecture: The impact of coupling on defect-related activity," *Journal of Systems and Software*, vol. 120, pp. 170-182, 2016.
- [14] T. Besker, H. Ghanbari, A. Martini, and J. Bosch, "The influence of Technical Debt on software developer morale," *Journal of Systems and Software*, vol. 167, 110586, 2020.
- [15] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *Journal of Systems and Software*, vol. 101, pp. 193-220, 2015.